

PDO

Base de datos

En un documento php dentro de la carpeta de lib que luego debemos linkear en cada uno de los otros doc con un `Include_once("libs/data_base.php");`

Conexión

Para abrir la conexión a través del método PDO es necesario establecer unas variables antes. En ellas se establecerá el nombre del servidor, el usuario y contraseña y el nombre de la base de datos.

```
$servername='db';
$dbname='donacion';
$username='root';
$password='test';
```

Una vez que esto se ha decidido, abrimos un try/catch para establecer la conexión y recoger el error en el caso de que suceda.

```
try{
    $connection = new PDO("mysql:host=$servername;dbname=$dbname",
$username,$password);
    $connection -> setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Database connected successfully";
    return $connection;
} catch (PDOException $e){
    echo "Connection failed: " . $e->getMessage();
}
```

Crear DB

```
function create_DB ($connection){
    $sql="CREATE DATABASE IF NOT EXISTS DONACION";
    $stmt=$connection->prepare($sql);
    $stmt->execute();
}
function select_DB($connection){
    $sql="USE donacion";
    $stmt=$connection->prepare($sql);
    $stmt->execute();
}
```

CRUD

INSERT

Preparamos el sql con una consulta preparada, después pasamos al stmt para preparar la conexión y el sql. Y ejecutamos el stmt con los parametros que necesitemos pasarle.

```
function register_donante($connection,$nombre,$apellido,$edad,$grp_sangre,$cod_postal,$tlf) {
    $sql="INSERT INTO
DONANTES(nombre,apellido,edad,grp_sangre,cod_postal,tlf)
VALUES(?,?,?,?,?,?)";
    $stmt=$connection->prepare($sql);
    $stmt->execute([$nombre,$apellido,$edad,$grp_sangre,$cod_postal,$tlf]);
}
```

SELECT

Para mostrar la informacion partimos de una situación similar al INSERT. Solo que al ejecutar no es necesario pasar ningún dato. Por último, en el return necesitamos transformar el stmt a través de un `fetch($stmt->fetchAll(PDO::FETCH_ASSOC))`.

```
function list_donantes($connecton) {
    $sql="SELECT * FROM DONANTES";
    $stmt=$connecton->prepare($sql);
    $stmt->execute();
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
}
```

UPDATE

Al igual que en ocasiones anteriores la prepaacion del stmt es igual, lo que cambia es la consulta del sql.

Lllamar la atención sobre el orden de los datos en la ejecución del stmt, **debe seguir el orden de las interrogaciones de la consulta**.

```
function update_donante($connection, $id,$nombre,$email) {
    $sql="UPDATE DONANTES SET nombre=?,email=? WHERE id=?";
    $stmt=$connection->prepare($sql);
    $stmt->execute([$nombre,$email,$id]);
}
```

DELETE

Continuamos preparando el stmt del mismo modo que una cosulta de select, lo único que cambia es el contenido del sql.

```
function delete_donante($connection,$id) {
    $sql="DELETE FROM DONANTE WHERE id=?";
    $stmt=$connection->prepare($sql);
    $stmt->execute([$id]);
}
```

Procesado en las páginas

Retroalimentación

En cada formulario que tengamos que realizar se validan los datos antes de que se envíen. Para ello, las **primeras líneas en el body** deben hacer referencia a esto:

```
<?php include_once ("style/header.html");
$mensaje="";
if($_SERVER['REQUEST_METHOD']=="POST" && isset($_POST['submit'])) {
    $nombre= validate($_POST['name']);
    $apellidos=validate($_POST['apellidos']);
    $age=validate($_POST['age']);
    $grp_sangre=validate($_POST['grp_sangre']);
    $cod_postal=validate($_POST['cod_postal']);
    $tlf=validate($_POST['tlf']);

    $connection = getConnection();
    select_DB($connection);
}
```

- En el caso de querer incluir cualquier efecto estético como puede ser `include_once ("style/header.html")`; debe realizarse antes también.
- Debemos chequear cuando carguemos la página si hay algo cargado con un `if`, lo normal es que en este momento este vacío. En el `if` filtramos por el tipo de formulario (get o post) y que se haya enviado.

```
if($_SERVER['REQUEST_METHOD']=="POST" && isset($_POST['submit']))
```

Para el primer paso (el tipo de formulario) usamos la variable `$_SERVER` para acceder al método de envío. Y con `isset($_POST)` sabemos si se ha activado el botón de submit.

En el caso de entrar en el `if` procedemos a validar todos los campos que queramos. Para ello podemos crear otro documento con todas las validaciones (también en libs), en este caso debemos incluir ese documento (antes de cualquier código). Cada campo validado se guarda en una variable que se reconozca para poder pasarlas a la DB.

```
$conexion = get_conexion();
select_DB($conexion);
```

```
alta_usuario($conexion, $nombre, $apellidos, $edad, $provincia);  
$conexion->close();
```

Primero abrimos conexión y la guardamos en la variable. Acto seguido llamamos a la función para conectarnos a la DB con esa conexión. Por último llamamos a la función que realice la petición a la DB que busquemos, pasándole la conexión y los datos validados. Siempre cerrar la conexión a la base de datos, se puede hacer a través de una función o en cada documento.

Una vez tengamos esta parte lista podremos empezar con el formulario. La parte de PHP de los formularios se centra en la primera línea, en la propia etiqueta del Form.

```
<Form method="post" action="<?php echo  
htmlspecialchars($_SERVER['PHP_SELF']);?>">
```

En ella se indica el método por el que se va a enviar el formulario. En la segunda parte se referencia el donde se va a enviar. Junto con `htmlspecialchars` para evitar problemas con código malicioso, y con la supervariable `$_SERVER`.

POO

Base de datos

En un documento php dentro de la carpeta de lib que luego debemos linkar en cada uno de los otros doc con un `Include_once("libs/data_base.php");`

Conexión

```
function get_conexion(){  
    $conexion = new mysqli('db','root','test');  
    if($conexion->connect_errno != null){  
        die("Fallo en la conexión: ".$conexion->connect_error." con número  
". $conexion->connect_errno);  
    }  
    return $conexion;  
}
```

En la primera línea le indicamos el sistema que se usará `new mysqli` y le pasamos la información de la DB (nombre, usuario y contraseña) `('db','root','test')`.

En el caso de que devuelva desde la variable un error, debemos recogerlo y mostrarlo. Con un `if`, cuya condición sea que el error sea diferente a nulo, hacemos que imprima el mensaje de error y que termine el evento usando `die`.

```
($conexion->connect_errno != null)
```

Crear DB

Para crear la DB debemos crear una función que pueda trabajar con las consultas a mysql. La estructura de esta función será usada para crear tablas, y otras consultas, solo cambiarán las variables que necesitan y la estructura interna.

```
function crear_DB($conexion) {  
    $sql = "Create database if not exists tienda";  
    ejecutar_consulta($conexion, $sql);  
}  
function select_DB($connection) {  
    $sql=" USE tienda";  
    $result = execute_query($connection,$sql);  
    return $result;  
}
```

Para crear esta función debemos facilitarle la conexión que hemos creada anteriormente. Una vez dentro crearemos una variable con la consulta deseada y llamaremos a nuestra función para ejecutarla.

Crear tablas

Es una creación estandar de mysql para una tabla dentro de la variable sql y después pasarla a la función de ejecución

```
function create_user_table($connection) {  
    $sql="CREATE TABLE IF NOT EXISTS USERS(  
    id_user INT (6) auto_increment primary key,  
    ...  
    provincia varchar(50) not null  
    )";  
    execute_query($connection,$sql);  
}
```

Función ejecutar consultas

```
function execute_query($connection, $sql) {  
    $result = $connection->query($sql);  
    if($result == false) {  
        die ("Error al procesar la petición: " . $connection->error);  
    }  
    return $result;  
}
```

Es necesario pasar la consulta y la conexión. Con la conexión ejecutamos el método query, pasándole la consulta. Y para manejar los errores con un if nos aseguramos que si el evento es igual a false debemos matar el proceso e imprimir el error.

CRUD

Al contrario que en PDO, aquí las consultas devuelven siempre una respuesta. La Preparación de la consulta (\$sql) es igual, la diferencia es que en POO nos permite crear una función que se centre en ejecutar cualquier petición CRUD. Por lo que los pasos de preparación del stmt se substituyen por la llamada a esta función.

Alerta, a la hora de crear el sql debemos entender que no usaremos las consultas preparadas, por ser más complejo de preparar en POO. Para ello ya indicaremos en la sección VALUES la correlación entre datos.

INSERT

```
function register_user($connection,$nombre,$apellido,$edad,$provincia) {  
    $sql="INSERT INTO USERS (nombre, apellidos, edad, provincia) VALUES  
    ('$nombre','$apellido', $edad, '$provincia')";  
    $stmt = execute_query($connection,$sql);  
    return $stmt;  
}
```

SELECT

```
function get_user($connection, $id_user){  
    $sql="SELECT * FROM USERS WHERE id_user=$id_user";  
    $result = execute_query($connection,$sql);  
    return $result;  
}
```

UPDATE

```
function edit_user($connection,  
$id_user,$nombre,$apellido,$edad,$provincia) {  
    $sql="UPDATE USERS SET nombre='$nombre', apellidos='$apellido',  
edad=$edad, provincia='$provincia' WHERE id_user=$id_user";  
    $result = execute_query($connection,$sql);  
    return $result;  
}
```

DELETE

```
function delete_user($connection, $id_user){  
    $sql="DELETE FROM USERS WHERE id_user=$id_user";  
}
```

```
$result = execute_query($connection,$sql);  
return $result;  
}
```

Cerrar conexion

Se recomienda Cerrar la sesión al final de cada consulta.

```
function close_connection($connection) {  
    $connection->close();  
}
```

Editar DB

En el caso de que queramos editar un elemento de nuestra base de datos, el usuario debe ver cargados los datos que ya tendríamos guardados para poder editarlos. En ese caso se añade una capa más de complejidad.

Para preparar la página inicial debemos generar una estructura php antes del código html. Debemos indicar y guardar en una variable la conexión a la DB, así como asignar variables vacías a todos los elementos que debe mostrar y editar el usuario.

Con una estructura if/else comprobamos si el usuario a mandado la información del registro. En el momento que se inicia esa página por primera vez esto va a ser negativo así que saltará al else. Pero una vez editado, sí que entrará en esta parte que es exactamente la misma que la de añadir una persona (guardar en variables los valores validados) y se guardan con nuestra función de editar_user.

¿Pero que pasa la primera vez que se carga la página? Al entrar al else se encuentra con este código:

```
if(isset($_GET["id"])) {  
    $id_user=$_GET["id"];  
    $user = buscar_usuario($conexion, $id_user);  
    if($user->num_rows >0) {  
        $row=$user->fetch_assoc();  
        $id_user=$row['id'];  
        $nombre=$row['nombre'];  
        $apellidos=$row['apellidos'];  
        $edad=$row['edad'];  
        $provincia=$row['provincia'];  
    }  
}else{  
    $id_user=0;  
    $nombre="";  
    $apellidos="";  
    $edad=0;  
    $provincia="corunha";  
}
```

En esta sección el if interior se asegura de recibir (por GET), en caso positivo busca con nuestra función el usuario y lo asigna a una variable. Con fetch dividimos por filas y reasignamos las variables del principio.

Por último, en el propio formulario, en el apartado valor le asignamos el homónimo.

SEGUNDO TRIMESTRE

Sesiones `$_SESSION`

Iniciar sesión

Para iniciar una sesión se requiere la sentencia `session_start()`; En el caso de que existan otras sesiones pero no se les pase, se creará una nueva sesión, si no hay ninguna sesión se accede a la supervariable `$_SESSION`. La sintaxis para crear una sesión similar a una variable normal: `$_SESSION["favcolor"] = "verde";`. Donde es necesario indicar que es una sesión y enter parentesis el nombre que se le asigna a dicha sesión:

```
// Start the session
session_start();
---
// Establecer variables de sesión
$_SESSION["favcolor"] = "verde";
$_SESSION["favanimal"] = "gato";
echo "Variables de sesión establecidas.";
```

Obtener sesiones

A la hora de acceder a la sesión debemos referir al nombre, pero siempre indicando `$_SESSION`:

```
echo "El color favorito es: " . $_SESSION["favcolor"] . "<br>";
echo "El animal favorito es: " . $_SESSION["favanimal"] . " ";
```

Todas las sesiones se almacenan en la variable global `$_SESSION`, por lo que una forma de acceder a todas las variables almacenadas sería:

```
print_r($_SESSION);
```

Se puede especificar que al obtener la información escojamos el id de la sesión: `echo 'A sesión actual é: '.session_id();`

Modificar/eliminar sesiones

Para modificar una sesión solo hay que sobreescribirla. En el caso de querer eliminarla la sintaxis es simple `unset($_SESSION["favcolor"]);`

Evitar ataques Session Fixation

Estos ataques se basan en los cuales es atacante registra una id en el server que luego se la pasa al usuario. Con esto en futuras ocasiones puede utilizar ese id para entrar como si fuera el usuario. Para evitar este tipo de ataques existen tres opciones:

Uso de cookies

En primer lugar podemos usar las cookies para evitar que la id viaje en la URL o en formularios. Para ello guardamos la session en alguna cookie y tendremos que recuperarla, esta situación reduce las posibilidades, pero nunca son cero. Con el siguiente código se hace obligatorio que la id de session solo se use si proviene de una cookie:

```
ini_set('session.use_only_cookies',1);
```

Marca de session

Para usar una marca de control debemos generarla primero, a la vez que iniciamos sesion creamos esta marca de la siguiente manera:

```
session_start();  
if (!isset($_SESSION['session_token'])) {  
    $_SESSION['session_token'] = bin2hex(random_bytes(32));  
}
```

Una vez que tengamos esta marca creada debemos entregársela al usuario, para ello podemos usar el propio formulario en el cual se loggea. Esto hará que podamos comparar las claves más adelante. En el caso que alguien pudiese introducir una session exterior no contaría con este campo. Para poder pasar esta clave incluiremos un campo oculto en el formulario:

```
<form action="procesar.php" method="post">  
    <input type="hidden" name="session_token" value="<?php echo  
htmlspecialchars($_SESSION['session_token']); ?>">  
    <!-- otros campos -->  
</form>
```

El tercer paso sería la comprobación de la marca, en ese caso se daría en la pagina que procese el loggeo. Donde con un if deberíamos primero confirmar la existencia de la marca y del envío, para posteriormente compararlos:

```
session_start();  
  
if (!isset($_SESSION['session_token'], $_POST['session_token']) ||  
    !hash_equals($_SESSION['session_token'], $_POST['session_token'])) {
```

```
// Token inválido: posible intento de session fixation o CSRF
die('Sesión no válida. Vuelve a iniciar sesión.');
```

Ya para terminar, para invalidar cualquier session exterior debemos regenerar la sesión del usuario una vez logeado, para ello usaremos: `session_regenerate_id(true)`.

Renovar al loggear

La marca es una capa que protege desde el momento del formulario. El caso de la renovacion del id es la capa ultima, la que anula la sesión introducida. Por ello puede usarse sola o en conjunto con la marca. Este método es el que se recomienda usar si o si, en conjunto con los otros.

```
if ($usuario_logueado === true) {
    session_regenerate_id(true);
    $_SESSION['logueado'] = true;
}
```

Cookies

Crear cookies

La sintaxis de una cookie es simple, y de los valores que pide el único que es obligatorio es el nombre, es resto son opcionales. **Para settear una cookie se usan paréntesis.** `setcookie(name, value, expire, path, domain, secure, httponly);`

Recuperar cookies

Para poder recuperar una cookie debemos usar la variable global `$_COOKIE`. El siguiente código muestra como recuperar una cookie, junto con el paso previo de comprobar si existe esa cookie. **Para recuperar la cookie se utiliza los corchetes para encapsular los nombres de las variables.**

```
<?php
$cookie_name = "usuario";
$cookie_value = "Sabela";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); //
86400 = 1 day
?>
<html>
<body class="<?php echo ($tema=='dark') ? 'bg-dark text-white' : 'bg-
light text-dark'; ?>">
<?php
if(!isset($_COOKIE[$cookie_name])) {
    echo "La cookie con nombre '" . $cookie_name . "' no está
definida!";
} else {
    echo "La cookie '" . $cookie_name . "' está definida!<br>";
    echo "Su valor es: " . $_COOKIE[$cookie_name];
}
```

```
}  
?>  
</body>  
</html>
```

Modificar y eliminar

Al igual que en la session para modificar una cookie solo es necesario reescribirla. En el caso de querer eliminar una cookie se modifica su fecha de caducidad y se autoelimina.

Ficheros

Abrir/leer

Para poder abrir un fichero tenemos dos opciones: con "echo" se reproduce el texto de manera muy simple, sin respetar saltos de página ni formatos. Con la función `fopen()` se permiten más opciones. Es una forma de crear una lista de parámetros que se apliquen a la hora de abrir el documento. Primero se guarda en una variable en la que le indicamos que archivo sobre el que queremos que se apliquen, y los parámetros (en este caso se añade un "die" en caso de que no sea posible no genere errores). Después se hace un "echo" junto con la función "fread()" (que aplica los parámetros anteriores). Dentro de esta, esta la variable con los parámetros y el archivo. Por último, se cierra con la función "fclose()" una vez que se acaba de trabajar con el.

```
$mifichero = fopen("webdictionary.txt", "r") or die("Unable to open  
file!");  
echo fread($mifichero, filesize("webdictionary.txt"));  
fclose($mifichero);
```

Los parámetros que se pueden añadir son:

- r/r+ - Solo de lectura o lectura/escritura con el cursor al principio.
- w/w+ - Solo escritura o lectura/escritura con el cursor al principio, y si no existe el fichero se crea. Que el cursor este al principio implica que si se escribe algo se sobrescribe el contenido.
- a/a+ - Solo escritura o lectura/escritura con el cursor al final, y si no existe el fichero se crea. Que el cursor esté al final significa que al escribir algo, esto se añade a lo ya escrito.

Otras funciones

- fgets() - Sirve para leer solo una línea del documento.
- fgetc() - Sirve para leer un solo carácter.
- feof () - Sirve para comprobar si se llegó al final de documento.

```
$mifichero = fopen("webdictionary.txt", "r") or die("Unable to open  
file!");  
// Output one line until end-of-file  
while(!feof($mifichero)) {  
    echo fgets($mifichero) . "<br>";  
}
```

```
}  
fclose($mifichero);
```

Crear

Para crear también se utiliza la función `fopen()` porque se asume que se abre para escribir o agregar (`$mifichero = fopen("testfile.txt", "w")`).

```
$mifichero = fopen("nuevoarchivo.txt", "w") or die("Unable to open  
file!");  
$txt = "Miguel\n";  
fwrite($mifichero, $txt);  
$txt = "Juan\n";  
fwrite($mifichero, $txt);  
fclose($mifichero);
```

En este código se abre un documento (al no existir se crea), con el parámetro de escritura y cursor al principio. Seguido de la inclusión de dos nombres. Si se repitiese este código con otros nombres se substituiría lo ya existente, sin embargo cambiando la "w" por "a" se añadirían.

Subir archivos

Para poder subir un archivo es necesario cumplir una serie de requisitos: que el formulario se envíe por método "post", debe tener el atributo `enctype="multipart/form-data"` y el input con el atributo `type = "file"` debe tener asociado un boton de "examinar" (normalmente los hace automaticamente el navegador).

Carga

En primer lugar guardamos la dirección en la que se guardaran los archivos y nos aseguramos de que tengan permiso de escritura. Tras esto empezamos a trabajar con el archivo, comprobamos (1º if) que el envio es por POST. El primer paso es comprobar si hay archivo

```
// Nombre del directorio donde se guardarán los archivos subidos  
$targetDir = __DIR__ . '/uploads/';  
  
// Asegurarse de que la carpeta existe y tiene permisos de escritura  
if (!is_dir($targetDir) && !mkdir($targetDir, 0777, true)) {  
    die('Error: no se puede crear la carpeta de destino.');
```

```
}  
  
// Nombre del campo input en el formulario  
$fieldName = 'fileToUpload';  
  
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    if (!isset($_FILES[$fieldName])) {  
        die('Error: no se ha enviado ningún archivo.');
```

```
}
```

```

    $file = $_FILES[$fieldName]; //SIMPLIFICAL EL ACCESO A LA
INFORMACIÓN
    if ($file['error'] !== UPLOAD_ERR_OK) { // Y NO $_FILES[$fieldName]
['error']
        die('Error en la subida: ' . $file['error']);
    }
    // Verificar tamaño máximo (por ejemplo, 500 KB)
    if ($file['size'] > 500000) {
        die('El archivo es demasiado grande.');
```

original del doc

```

    }
    // Nombre seguro del archivo y ruta destino
    $fileName = basename($file['name']); //elimina una posible ruta
    $targetFile = $targetDir . $fileName;

    // Extensión del archivo en minúsculas
    $fileExtension = strtolower(pathinfo($targetFile, PATHINFO_EXTENSION));

    // Extensiones permitidas (ejemplo)
    $allowedExtensions = ['jpg', 'jpeg', 'png', 'gif', 'txt', 'pdf'];
    if (!in_array($fileExtension, $allowedExtensions, true)) {
        die('Tipo de archivo no permitido. Solo se aceptan: ' . implode('
', $allowedExtensions));
    }
    // Comprobar si el archivo ya existe
    if (file_exists($targetFile)) {
        die('El fichero ya existe.');
```

original del doc

```

    }
    // Mover el archivo desde la carpeta temporal a la carpeta uploads
    if (!move_uploaded_file($file['tmp_name'], $targetFile)) {
        die('Error al mover el archivo al directorio de destino.');
```

original del doc

```

    }
    echo '<!DOCTYPE html>';
    echo '<html lang="es"><head><meta charset="UTF-8">
<title>Resultado</title></head><body>';
    echo '<h1>Archivo subido correctamente</h1>';
    echo '<p>Nombre: ' . htmlspecialchars($fileName, ENT_QUOTES, 'UTF-8') .
'</p>';
    echo '<p>Tamaño: ' . htmlspecialchars($file['size'], ENT_QUOTES, 'UTF-
8') . ' bytes</p>';
    echo '<p>Tipo: ' . htmlspecialchars($file['type'], ENT_QUOTES, 'UTF-8')
. ' </p>';
    echo '<p><a href="formulario.php">Volver al formulario</a></p>';
    echo '</body></html>';
} else {
    die('Acceso inválido. Usa el formulario para subir el archivo.');
```

Filtrado de datos

Existen varias formas de filtrado de datos: validacion (Ej para email: `FILTER_VALIDATE_EMAIL`) o saneamiento (Ej para email: `FILTER_SANITIZE_EMAIL`). En el primer caso, si la información proporcionada

no cumple con lo que se pide no se tomará, en el segundo se eliminan aquellos elementos que no se permitan. en el caso de los arrays se aplica una sintaxis mas compleja:

```
filter_var_array(array $data, array|int $args, bool $add_empty = true):  
array|false|null
```

Un ejemplo con todos los filtros aplicados sería:

```
$data = [  
    'nombre' => 'Iván Gómez',  
    'email' => 'ivan@example.com',  
    'edad' => '18'  
];  
$args = [  
    'nombre' => FILTER_SANITIZE_STRING, // Elimina caracteres dañinos de la  
cadena  
    'email' => FILTER_VALIDATE_EMAIL, // Valida que el email sea válido  
    'edad' => [  
        'filter' => FILTER_VALIDATE_INT, // Valida que sea un entero  
        'options' => ['min_range' => 18, 'max_range' => 65], // Rango  
permitido  
    ]  
];  
$result = filter_var_array($data, $args);  
print_r($result);
```

Variables de entorno

Este tipo de variables afectan a dos ficheros docker-compose.yml y .env, y buscan guardar cierto tipo de informacion para que sea igual en todas las construcciones. En el docker-compose.yml las variables que se agrupan dentro de "enviroment" se transferirán a .env, donde no precisa ninguna estructura específica (un simple corta y pega).

```
services:  
  www:  
    build: .  
    ports:  
      - "80:80"  
    volumes:  
      - ./www:/var/www/html  
    links:  
      - db  
    networks:  
      - default  
    extra_hosts:  
      - "host.docker.internal:host-gateway"  
    environment:
```

```
APP_ENV: development
APP_DEBUG: "true"
DATABASE_HOST: db
DATABASE_USER: colegio
DATABASE_PASSWORD: colegio
```

Siendo sustituido por:

```
services:
  www:
    build: .
    ports:
      - "80:80"
    volumes:
      - ./www:/var/www/html
    env_file:
      - .env
```

Login y sesiones

Para poder tener cierto control sobre los usuarios registrados usaremos un pequeño formulario para luego comprobar con los usuarios que tengamos registrados.

```
session_start();
function comprobar_usuario($nombre, $pass)
{
    if($nombre == "usuario" && $pass="abc123.") {
        $usuario['nombre']="usuario";
        $usuario['rol']=0;
        return $usuario;
    }elseif($nombre == "admin" && $pass="1234") {
        $usuario['nombre']="admin";
        $usuario['rol']=1;
        return $usuario;
    }else{
        return false;
    }
}
```

```
//Comprobar si se reciben los datos
if($_SERVER["REQUEST_METHOD"]=="POST") {
    $usuario = $_POST["usuario"];
    $pass = $_POST["pass"];
    $user = comprobar_usuario($usuario, $pass );
    if(!$user) {
        $error = true;
    }else{
```

```
$_SESSION['usuario'] = $user;
//Redirigimos a index.php
header('Location: index.php');
}
}
```

En el caso de querer eliminar la sesión con el fin de cerrarla o cambiarla debemos destruirla

```
session_start();
$_SESSION = array();
session_destroy();
header("Location: index.php");
```

Clases y objetos

Clase y objeto

Los objetos son instancias de una clase, y las clases son estructuras base para la creación de objetos. Esta estructura básica tiene una serie de puntos obligatorios: Para empezar debe declararse como `class` y debe ser nombrada en mayuscula. Lo primero que encontramos en la clase son las variables que formarán el objeto, para que no se pueda acceder a estas variables debemos usar los setters y getters.

```
class Fruit {
    // Propiedades
    public $name;
    private $color;
    // Métodos
    function setName($name) {
        $this->name = $name;
    }
    function getName() {
        return $this->name;
    }
}
```

Una vez generada la estructura del objeto debemos crear el objeto, para ello debemos indicarlo `$banana=new Fruit();` y luego pasar los valores de ese objeto `$banana->setName('Banana');` El acceso a estas variables se puede limitar tambien a mayores de los setter y es que podemos generar variables que solo se usen en la propia clase. Estas limitaciones se pueden aplicar tambien a las funcinoes.

- public: se puede acceder a la propiedad o método desde cualquier lugar.
- protected: se puede acceder a la propiedad o método dentro de la clase y por clases derivadas de esa clase (herencia, por ejemplo).
- private: Solo se puede acceder a la propiedad o método dentro de la clase.

`$this` & `$instanceof`

`$this` es usado cuando quieres acceder a una de las variables del propio objeto con el que estas trabajando. `$this->name=$name`. En el caso de `("x" instanceof "x")` se utiliza para comprobar si un elemento es un objeto de una clase.

Constructores y destructores

El constructor es la función que crea de por si los objetos siguiendo las variables necesarias. Su posición dentro de la estructura suele ser debajo de la declaración de las variables.

```
//Función de constructor al que le pasamos el nombre y lo iguala en el
objeto.
function __construct($name) {
    $this->name = $name;
}
```

En la posición contraria tenemos al destructor, para poder borrar el objeto que no vamos a usar más. Su posición suele ser inmediatamente posterior al constructor.

```
function __destruct() {
    echo "The fruit is {$this->name}.";
}
```

Herencia

Llamamos herencia cuando una clase deriva de una superior. Esto también conlleva que las variables, funciones y métodos que se declarasen en la clase superior se pueden usar en la nueva clase. Para indicar que una clase hereda de otra se modifica el enunciado de la clase `class Dog extends Animal`. Estos métodos heredados se pueden redefinir si nombras a un método de la misma manera. De la misma manera podemos proteger una clase para que no pueda tener clases heredadas, al nombrar una clase con `final class Animal`. Está podrá heredar de clases superiores pero no podran existir clases por debajo de ella.

Constantes de clase

Pos lo mismo que fuera de las clases, pero ahora tienen una sintaxis ligeramente diferente para declarar y para llamarlas.

```
class Goodbye {
    const LEAVING_MESSAGE = "Adiós, nos vemos pronto!";
    public function goodbye() {
        echo self::LEAVING_MESSAGE;
    }
}
```

Clases abstractas

Las clases abstractas son aquellas que contienen al menos un método abstracto, no se pueden instanciar de estas clases, es necesario crear una clase hija para ello.

```
abstract class ParentClass {
    abstract public function someMethod1();
    abstract public function someMethod2($name, $color);
    abstract public function someMethod3() : string;
}
```

En el caso de la clase que hereda debe de redefinir todos los métodos que aparezcan como abstractos, y estos deben tener el mismo modificador de acceso o uno menos restrictivo.

Interfaces & traits

La interfaz se puede considerar una variable de las clases abstractas. Estas no pueden tener un método o función concretos y solo pueden ser públicos. La utilidad de estas limitaciones es que las clases que hereden de ella no se ven restringidas a solo heredar de una interfaz. En cambio los traits son algo más sencillo, mientras que las interfaces obligan a redefinir los métodos, los traits no es necesario. A cambio tienes que definirlos en el trait. Los traits también permiten el uso/herencia de varios de ellos a la vez en una clase secundaria, para indicarle que utilice los traits debemos indicarlo en la primera línea antes de las variables.

```
class Persona {
    use traitA, traitB, traitC;
    // ♦ Propiedades
    private $nombre;
    private $edad;
    // ♦ Constructor...
```

Un ejemplo con herencia, interface y traits:

```
interface Identificable {
    public function getId(): int;
    public function getNombre(): string;
}

trait Saludable {
    public function respirar(): string {
        return "Respirando...";
    }
    public function dormir(): string {
        return "Durmiendo...";
    }
}

class Humano {
    protected $edad;

    public function __construct(int $edad) {
        $this->edad = $edad;
    }
}
```

```

    }
    public function caminar(): string {
        return "Caminando";
    }
    public function getEdad(): int {
        return $this->edad;
    }
}

class Persona extends Humano implements Identificable {
    use Saludable;

    private $id;
    private $nombre;

    public function __construct(int $id, string $nombre, int $edad) {
        parent::__construct($edad);
        $this->id = $id;
        $this->nombre = $nombre;
    }
    public function getId(): int {
        return $this->id;
    }
    public function getNombre(): string {
        return $this->nombre;
    }
    public function presentarse(): string {
        return "Hola, soy " . $this->nombre . " y tengo " . $this->edad . "
años.";
    }
}

// Uso
$persona = new Persona(1, "Lucía", 30);

echo $persona->presentarse(); // Hola, soy Lucía y tengo 30 años.
echo $persona->caminar();     // Caminando
echo $persona->respirar();    // Respirando...
echo $persona->dormir();      // Durmiendo...

```

Estáticos

Los métodos estáticos de las clases se indican en la declaración del propio método `public static function saludo()` se puede acceder a ellos desde cualquier parte, sin tener que crear una instancia `Saludar::saludo()`; . Para poder acceder a ellos desde la propia clase debemos cambiar la sintaxis por un `self::saludo()`; . Se pueden usar estos métodos desde otras clases sin tener que implementar `extend` o usar nada.

Objetos vacíos

Es un método para crear un objeto sin tener una estructura creada de antemano. Es especialmente útil para enviar información entre sistemas a través de JSON. Es una clase en particular `stdClass()`

```
$obj = new stdClass();  
$obj->name = "Nick";  
$obj->surname = "Doe";  
$obj->age = 20;  
$obj->address = "Santiago de Compostela. A Coruña";
```

Errores y excepciones

Al igual que se vio con anterioridad se pueden manejar las excepciones de dos maneras, generando nosotros la excepción con el throw y recogiendo las que se originen con una estructura try-catch. Podemos generar las excepciones personalizadas generando una clase propia que extienda de la clase Exception.

```
class DivisionPorCeroException extends Exception {  
    private $valorOriginal;  
  
    public function __construct($mensaje, $valorOriginal, $codigo = 0,  
Exception $anterior = null) {  
        // Llamar al constructor base  
        parent::__construct($mensaje, $codigo, $anterior);  
  
        // Guardar el valor original  
        $this->valorOriginal = $valorOriginal;  
    }  
    // Método para obtener el valor original  
    public function obtenerValorOriginal() {  
        return $this->valorOriginal;  
    }  
}  
function inverso($x) {  
    if (!$x) {  
        // Lanzar una excepción personalizada  
        throw new DivisionPorCeroException('Intento de división por cero.',  
$x);  
    }  
    return 1 / $x;  
}  
try {  
    echo inverso(5) . "\n";  
    echo inverso(0) . "\n"; // Esto lanzará la excepción  
} catch (DivisionPorCeroException $e) {  
    // Capturar la excepción personalizada  
    echo 'Excepción capturada: ', $e->getMessage(), "\n";  
    echo 'Valor original: ', $e->obtenerValorOriginal(), "\n";  
} finally {  
    // Bloque opcional que siempre se ejecuta  
    echo "Finalizando la operación.\n";  
}
```

TERCER TRIMESTRE

Flight

```
Flight::route('/saludar', function () {  
    echo 'Hola, bienvenido al módulo de DWCS!';  
});
```

Esta orientado a la creación de servicios web, para una mejor comunicación máquina-máquina que el código estándar. Para ello se requiera la importación de la librería básica de Flight a través de:

```
//Incluimos la librería de flight  
require 'flight/Flight.php';  
  
...  
  
//Iniciamos el servicio  
Flight::start();
```

Se utilizan, practicamente de manera exclusiva, en el conexiones a DB, por lo que hemos de aprender a abrir una DB y a manejarla (GET, POST, PUT y DELETE).

Abrir conexión

Para abrirla es tan sencillo como adaptar los métodos de PDO y POO a la estructura de Flight. Esta empieza siempre con `Flight::`, en el caso de querer abrir la conexión deberíamos usar el método `register`. Posteriormente le indicamos el nombre de la DB que queremos abrir (se usará luego como "método" para acceder a la DB), seguido del formato de conexión (PDO o POO) y por ultimo un array con los parámetros necesarios para una conexión PDO.

```
Flight::register('db', 'PDO', array('mysql:host=db;dbname=agenda', 'root', 'test'));
```

Acceso

En el caso de querer acceder a los datos debemos usar el método `Flight::route`, e indicarle con GET la acción y la búsqueda (formato ruta). Seguido la función que se encargue de la conexión. Dentro de esta función debemos preparar el statement indicando la DB (`Flight::db`) y al final para poder mostrar los resultado siempre usar el `Flight::json(...)`

```
Flight::route('GET /contactos', function(){  
    /*Codigo para recuperar contactos del usuario*/  
    $stmt=Flight::db()->prepare('SELECT * from contactos');
```

```
$stmt->execute();  
$contactos=$stmt->fetchAll();  
Flight::json($contactos);  
});
```

En el caso de querer buscar un elemento más específico como un id en concreto, dentro de la función hay que añadir una primera línea.

```
$id=Flight::request()->data->id;
```

Insertar

Al igual que en los otros casos empezamos con `Flight::route()` e indicamos que insertamos datos con POST. dentro de la función es donde cambia, con la petición de los datos que queramos guardar a través del método `request`. El resto es una función estandar de inserción.

```
Flight::route('POST /contactos', function(){  
    /*Codigo para añadir contacto*/  
    $nombre=Flight::request()->data->nombre;  
    $telefono=Flight::request()->data->telefono;  
    $email=Flight::request()->data->email;  
    $stmt=Flight::db()->prepare('INSERT INTO  
contactos(nombre,telefono,email) values(:nombre,:telefono,:email)');  
    $stmt->execute(['nombre'=>$nombre, 'telefono'=>$telefono,  
'email'=>$email]);  
});
```

Borrar

Indicamos un `Flight::route` y un DELETE con la información. Dentro de la función recuperamos el id con `Flight::request` y el resto es la función estandar de DB para eliminar registros.

```
Flight::route('DELETE /contactos/@id',function(){  
    /**codigo para eliminar usuario */  
    /**Recuperamos el id enviado por el json */  
    $id=Flight::request()->data->id;  
  
    /**Realizamos la peticion sql */  
    $stmt=Flight::db()->prepare('DELETE FROM contactos WHERE id=?');  
    $stmt->bindParam(1, $id);  
    $stmt->execute();  
});
```

Actualizar

Indicamos un `Flight::route` y un PUT con la información. Dentro de la función recuperamos los datos con `Flight::request` y el resto es la función estándar de DB para actualizar registros.

```
Flight::route('PUT /contactos/@id',function(){
  /*Codigo para actualizar*/

  /**Recuperamos los datos con los que trabajaremos */
  $id=Flight::request()->data->id;
  $nombre=Flight::request()->data->nombre;
  $telefono=Flight::request()->data->telefono;
  $email=Flight::request()->data->email;

  /**Realizamos la petición sql */
  $stmt=Flight::db()->prepare('UPDATE contactos SET nombre=:nombre,
telefono=:telefono, email=:email WHERE id=:id');
  $stmt->execute(['nombre'=>$nombre, 'telefono'=>$telefono,
'email'=>$email, 'id'=>$id]);
});
```